

Database Tables - Quick Reference

Table Relationships Summary

USER MANAGEMENT

users (1) —→ (N) user_preferences
users (1) —→ (N) user_interests
users (1) —→ (N) trips
users (1) —→ (N) poi_user_ratings
users (1) —→ (N) itinerary_feedback

POINTS OF INTEREST

pois (1) —→ (N) poi_categories
pois (1) —→ (N) poi_opening_hours
pois (1) —→ (N) poi_photos
pois (1) —→ (N) cluster_pois
pois (1) —→ (N) itinerary_pois
pois (1) —→ (N) itinerary_restaurants
pois (1) —→ (N) poi_user_ratings

poi_opening_hours (1) —→ (N) poi_time_windows

TRAVEL & DISTANCE

pois (origin) —┐
 ├──→ travel_times (N:N relationship)
pois (destination) —┘

TRIP PLANNING

destinations (1) —→ (N) trips
users (1) —→ (N) trips
trips (1) —→ (N) clusters
trips (1) —→ (N) itineraries
trips (1) —→ (N) ga_executions
trips (1) —→ (N) poi_user_ratings

CLUSTERING (Module III)

trips (1) —→ (N) clusters [one per day]
clusters (1) —→ (N) cluster_pois
pois (1) —→ (N) cluster_pois

ITINERARY OPTIMIZATION (Module IV)

trips (1) —→ (N) itineraries
clusters (1) —→ (N) itineraries
itineraries (1) —→ (N) itinerary_pois [ordered sequence]
itineraries (1) —→ (N) itinerary_restaurants
pois (1) —→ (N) itinerary_pois
pois (1) —→ (N) itinerary_restaurants
itineraries (1) —→ (N) itinerary_feedback

Core Tables by Importance

Critical Tables (Core System)

1. **users** - All users start here
2. **pois** - Central POI data
3. **trips** - Links users to their travel plans
4. **travel_times** - Required for GA optimization
5. **itineraries** - Final output to users

Required for Algorithm (Module III & IV)

6. **clusters** - K-means output
7. **cluster_pois** - POIs grouped by day
8. **itinerary_pois** - Optimized visit order
9. **poi_opening_hours** - Time constraints

Enhancement Tables

10. **user_preferences** - Personalization
11. **user_interests** - NLC-detected interests
12. **poi_categories** - POI classification
13. **itinerary_restaurants** - Lunch suggestions

Evaluation Tables

14. **poi_user_ratings** - User ratings
15. **itinerary_feedback** - Survey responses (P1-P16)
16. **ga_executions** - Performance metrics

Supporting Tables

17. **destinations** - City data
18. **poi_photos** - Visual content

19. **poi_time_windows** - Complex schedules

20. **api_cache** - Performance optimization

Workflow Data Flow

1. User Creates Trip

Input:

- User selects destination
- Chooses dates (start_date, end_date)
- Sets daily schedule (start time, end time, lunch time)

Tables Updated:

- trips (new row)

2. System Gets POIs

Process:

- Fetch POIs from Google Places API
- Filter by user preferences and interests
- Calculate popularity scores

Tables Updated:

- pois
- poi_categories
- poi_opening_hours
- poi_photos
- api_cache (store responses)

3. Calculate Travel Times

Process:

- Get distance/time between all POI pairs
- Use Google Distance Matrix API

Tables Updated:

- travel_times (N×N matrix)
- api_cache

4. K-means Clustering (Module III)

Input:

- Selected POIs for the trip
- Number of days

Process:

- Group POIs by geographical proximity
- One cluster per day

Tables Updated:

- clusters (one per day)
- cluster_pois (POI assignments)

5. Genetic Algorithm Optimization (Module IV)

Input:

- Cluster POIs for one day
- Travel times matrix
- POI opening hours
- Time constraints (daily schedule, lunch time)

Process:

- Initialize population of random routes
- Evolve solutions over generations
- Select best fitness score

Tables Updated:

- itineraries (best solution + alternatives)
- itinerary_pois (ordered visits)
- itinerary_restaurants (lunch options)
- ga_executions (performance log)

6. User Views & Provides Feedback

Process:

- User sees recommended itinerary
- Visits POIs during trip
- Rates experience

Tables Updated:

- poi_user_ratings (POI ratings)
- itinerary_feedback (survey responses P1-P16)

Query Examples

Get user's complete itinerary for a day

sql

```
SELECT * FROM v_itinerary_details
WHERE trip_id = ? AND day_number = ?
ORDER BY visit_order;
```

Find POIs open at specific time

```
sql

SELECT p.* FROM pois p
JOIN poi_opening_hours oh ON p.poi_id = oh.poi_id
WHERE oh.day_of_week = ?
      AND oh.open_time <= ?
      AND oh.close_time >= ?
      AND oh.is_closed = FALSE;
```

Get travel time between two POIs

```
sql

SELECT duration_seconds, distance_meters
FROM travel_times
WHERE origin_poi_id = ?
      AND destination_poi_id = ?
      AND travel_mode = 'walking';
```

Get best itinerary for a trip day

```
sql

SELECT * FROM itineraries
WHERE trip_id = ?
      AND day_number = ?
      AND is_best_solution = TRUE;
```

Calculate user satisfaction metrics

```
sql

SELECT
  AVG(understand_me_p1) as understand_avg,
  AVG(accuracy_p4) as accuracy_avg,
  AVG(novelty_p11) as novelty_avg,
  AVG(attitude_p15) as attitude_avg
FROM itinerary_feedback
WHERE user_id = ?;
```

Foreign Key Cascade Behavior

ON DELETE CASCADE

When a parent record is deleted, child records are automatically deleted:

- Delete **user** → deletes all user's preferences, interests, trips, ratings
- Delete **trip** → deletes all clusters, itineraries, GA executions
- Delete **poi** → deletes all categories, photos, opening hours, ratings
- Delete **cluster** → deletes all cluster_pois
- Delete **itinerary** → deletes all itinerary_pois and feedback

No CASCADE (keeps children)

These relationships preserve child records:

- Delete **destination** → trips remain but destination_id becomes NULL
-

Performance Considerations

Heavily Queried Tables

1. **pois** - Almost every query needs POI data
2. **travel_times** - Accessed thousands of times during GA
3. **itinerary_pois** - Frequently read to display routes

Large Tables

1. **travel_times** - $O(N^2)$ where N = number of POIs
2. **api_cache** - Grows with API usage
3. **poi_opening_hours** - 7 rows per POI minimum

Index Strategy

- Primary keys: automatic indexes
- Foreign keys: automatic indexes
- Location searches: index on (latitude, longitude)
- Time-based queries: index on date fields
- Text searches: consider full-text index on POI names

Caching Strategy

- API responses cached for 24 hours
- Travel times cached indefinitely (rarely change)
- Regular cleanup of expired cache entries